



# UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

---

## FACULTAD DE INGENIERÍA

### Diseño Digital Moderno

**Tarea:** Tarea 1: Sistemas Numéricos, Conversiones y Aritmética

**Profesor:** Dr. Oscar Francisco Fuentes Casarrubias

**Semestre:** 2027-1

**Integrantes:** Duarte Puertas Manuel

Hernandez Rosas Jafet Daniel

Reyes Garcia Miguel Angel

Torres Rodriguez Lizeth Danae

**Grupo:** 5

**Fecha de entrega:** 3 de septiembre de 2026



# 1 Introducción

El primer problema consiste en desarrollar un conversor que acepte números reales (incluidos negativos y fraccionarios) en bases del 2 al 16, validando que cada carácter sea válido para la base de origen. El algoritmo debe interpretar el número con notación posicional, usar divisiones sucesivas para la parte entera y multiplicaciones sucesivas para la parte fraccionaria, y no recurrir a funciones predefinidas de conversión del lenguaje.

El segundo problema requiere emular el comportamiento de un sumador binario con signo de 6 bits (1 bit de signo y 5 de magnitud), implementando una función modular de suma de 1 bit que use exclusivamente operaciones aritméticas básicas, y aplicando Complemento a 2 para representar números negativos.

Ambos problemas se resolvieron en un único programa de Python con interfaz gráfica (Tkinter), guardado en el archivo `tarea1.py`: una pestaña para la conversión de bases y otra para el sumador binario. La conversión usa aritmética exacta con fracciones, de modo que el único recorte de precisión ocurre al final, al limitar la salida a 16 dígitos fraccionarios.

## 2 Desarrollo

### 2.1 Problema 1: Conversor de Bases Numéricas

#### 2.1.1 Algoritmo de conversión

La conversión se divide en dos etapas. La primera interpreta el número de entrada con notación posicional y lo convierte a un valor exacto representado como `Fraction`. La parte entera se recorre de izquierda a derecha, multiplicando el acumulador por la base y sumando el valor del dígito actual. Por ejemplo, para convertir  $1011_2$  a decimal:

$$(((1 \times 2 + 0) \times 2 + 1) \times 2 + 1) = 11_{10}$$

La parte fraccionaria se acumula como la suma de las fracciones  $d_k/base^k$ , donde  $d_k$  es el valor posicional del dígito en la posición  $k$  después del punto decimal. Al usar `Fraction`, el valor intermedio es exacto y no se pierde precisión por punto flotante.

La segunda etapa genera el número en la base destino. Para la parte entera se usan divisiones sucesivas: se divide el número entre la base, el residuo es el dígito menos significativo, y el cociente se repite hasta llegar a cero. Para la parte fraccionaria se multiplica la fracción por la base destino; la parte entera del resultado es el siguiente dígito, y la nueva fracción se repite hasta la precisión deseada (16 dígitos). Por ejemplo, para convertir  $0.625_{10}$  a binario:

$$\begin{aligned} 0.625 \times 2 &= 1.25 && \rightarrow && 1 \\ 0.25 \times 2 &= 0.5 && \rightarrow && 0 \\ 0.5 \times 2 &= 1.0 && \rightarrow && 1 \\ \Rightarrow 0.625_{10} &= 0.101_2 \end{aligned}$$

#### 2.1.2 Validación de entrada

Antes de procesar los números, la función `validar_numero()` verifica que cada carácter sea válido para la base de origen: si la base es 9 solo se aceptan dígitos del 0 al 8; si la base es 2 solo se permiten 0 y 1. La validación también admite un signo negativo únicamente al inicio, permite a lo más un punto decimal y rechaza cadenas vacías o que terminen en punto. La

interfaz muestra el error con un cuadro de diálogo cuando el número no corresponde a la base seleccionada.

### 2.1.3 Implementación

Lo implementamos en Python con las siguientes funciones:

- `validar_numero()`: verifica que cada dígito sea válido para la base origen.
- `a_fraccion()`: interpreta el número con notación posicional y lo convierte a un valor exacto Fraction, incluyendo el signo negativo.
- `fraccion_a_base()`: convierte el valor a la base destino con divisiones sucesivas (entera) y multiplicaciones sucesivas (fraccionaria), limitando la salida a 16 dígitos.
- `convertir()`: une las dos etapas: interpreta la base origen y genera la base destino.

## 2.2 Problema 2: Emulador de Sumador Binario de 6 bits

### 2.2.1 Función de suma de 1 bit

La unidad básica del sumador es una función modular `sumar_un_bit()` que recibe dos bits y un acarreo de entrada, y produce un bit de resultado y un acarreo de salida. El algoritmo es el siguiente:

1. Calcula  $suma\_total = bit\_a + bit\_b + acarreo\_in$  (resultado entre 0 y 3).
2. El bit de resultado es  $suma = suma\_total \bmod 2$ .
3. El acarreo de salida es  $acarreo\_out = 1$  si  $suma\_total \geq 2$ , o 0 en caso contrario.

La misma función se reutiliza tanto para sumar 1 a los bits invertidos (en el Complemento a 2) como para cada una de las seis columnas de la suma completa, lo que mantiene la implementación modular.

### 2.2.2 Complemento a 2

Para representar números negativos en 6 bits (1 bit de signo y 5 de magnitud), aplicamos *Complemento a 2*:

1. Convierte la magnitud a su equivalente binario de 5 bits.
2. Invierte cada bit (1 por 0 y 0 por 1).
3. Suma 1 al bit menos significativo, reutilizando el sumador de 1 bit.

Por ejemplo, para  $-5$ : magnitud  $5 = 00101_2$ , invertir  $= 11010_2$ , sumar  $1 = 11011_2$ . Con bit de signo:  $111011_2$ .

El programa maneja por separado el caso límite  $-32$ : su magnitud de 5 bits es  $10000_2$ , al invertir se obtiene  $01111_2$  y al sumar 1 se vuelve  $10000_2$ , de modo que en 6 bits  $-32$  se representa como  $100000_2$ , que es su propio complemento.

### 2.2.3 Algoritmo de suma de 6 bits

La suma se realiza en 6 pasos modulares con la función `sumar_6_bits()`:

1. **Ciclos 1 a 5 (magnitud):** Se suman los bits de magnitud de derecha a izquierda, propagando el acarreo entre columnas.
2. **Ciclo 6 (signo):** Se suman los bits de signo junto con el acarreo que quedó del ciclo de magnitud.
3. **Detección de overflow:** Se compara el acarreo que entró a la celda de signo con el que salió. Si difieren, ocurrió desbordamiento aritmético y la interpretación decimal del resultado no representa la suma matemática.

El resultado de 6 bits se interpreta como decimal restando 64 cuando el bit de signo es 1. Por ejemplo,  $13 + (-5)$ :

$$001101_2 + 111011_2 = 001000_2 = 8_{10}$$

El acarreo que entra a la celda de signo (1) coincide con el que sale (1), por lo que no hay overflow.

### 2.2.4 Implementación e interfaz gráfica

Lo implementamos en Python con las siguientes funciones:

- `sumar_un_bit()`: función modular de suma de 1 bit.
- `decimal_a_binario_5_bits()`: convierte un entero del 0 al 31 a su magnitud de 5 bits.
- `sumar_uno_5_bits()`: suma 1 a una cadena de 5 bits reutilizando el sumador de 1 bit.
- `decimal_a_complemento_2_6_bits()`: genera la representación con signo de 6 bits y guarda la magnitud, los bits invertidos y el resultado después de sumar 1 para mostrarlos en el procedimiento.
- `sumar_6_bits()`: ejecuta la suma de 6 columnas (5 de magnitud + 1 de signo), propagando el acarreo y detectando overflow.

La interfaz gráfica se construyó con Tkinter y el tema `ttk "clam"`: una pestaña **Conversión** con los campos de número, base de origen y base de destino, y una pestaña **Sumador** con los dos enteros en el rango  $-32$  a  $31$ . El resultado y el procedimiento completo (magnitud, bits invertidos, suma columna por columna y acarreos) se muestran en una caja de texto de solo lectura.

## 3 Ejecución del programa

Para comprobar el funcionamiento del programa se ejecutaron los casos posibles de la interfaz: cinco en la pestaña de conversión y siete en la del sumador binario.

### 3.1 Casos de conversión

La Figura 1 muestra una conversión válida de un número entero en base 2 a decimal; la Figura 2 muestra el mismo proceso con parte fraccionaria, y la Figura 3 con dígitos alfabéticos. Las Figuras 4 y 5 muestran los mensajes de error cuando un dígito o una letra no pertenecen a la base seleccionada.

The screenshot shows a web application interface for base conversion. At the top, there are two tabs: 'Conversión' (active) and 'Sumador'. Below the tabs, the title 'Conversión entre bases' is centered. The form contains three input fields: 'Número:' with the value '101101', 'Base de origen:' with the value '2', and 'Base de destino:' with the value '10'. A green 'Convertir' button is positioned below these fields. Below the main form, there is a section titled 'Resultado y procedimiento' which displays the conversion details: 'Conversión de bases', 'Número: 101101', 'Base de origen: 2', 'Base de destino: 10', and 'Resultado: 45 (base 10)'.

Figura 1: Conversión válida de un número entero:  $101101_2 = 45_{10}$ .

Conversión

Sumador

### Conversión entre bases

Número:

Base de origen:

Base de destino:

#### Resultado y procedimiento

Conversión de bases

Número: 2AF  
Base de origen: 16  
Base de destino: 10

Resultado: 687 (base 10)

Figura 2: Conversión válida con parte fraccionaria:  $10.625_{10} = 1010.101_2$ .

Conversión

Sumador

### Conversor y calculadora de bases

Conversiones de bases y sumador binario de 6 bits en complemento a 2

### Conversión entre bases

Número:

Base de origen:

Base de destino:

#### Resultado y procedimiento

Conversión de bases

Número: 10.625  
Base de origen: 10  
Base de destino: 2

Resultado: 1010.101 (base 2)

Figura 3: Conversión válida usando letras:  $2AF_{16} = 687_{10}$ .

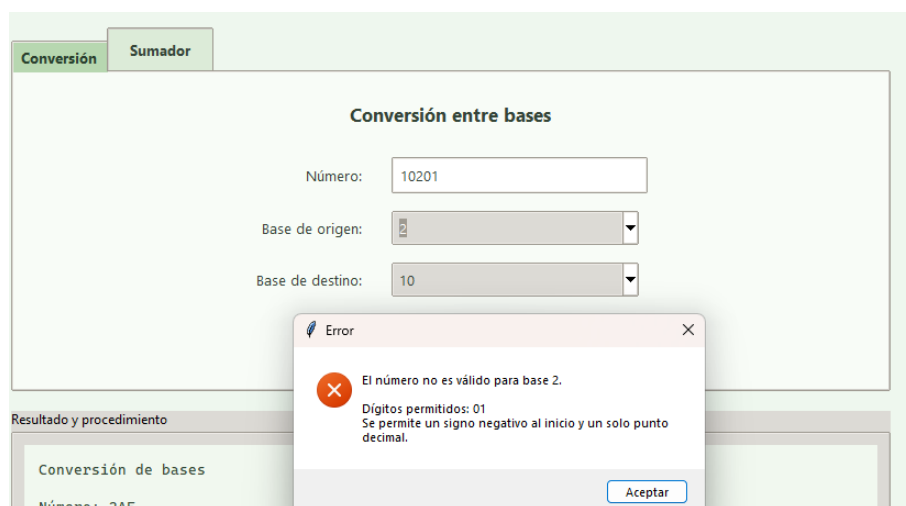


Figura 4: Validación de un dígito inválido: el número 10201 no es válido en base 2.

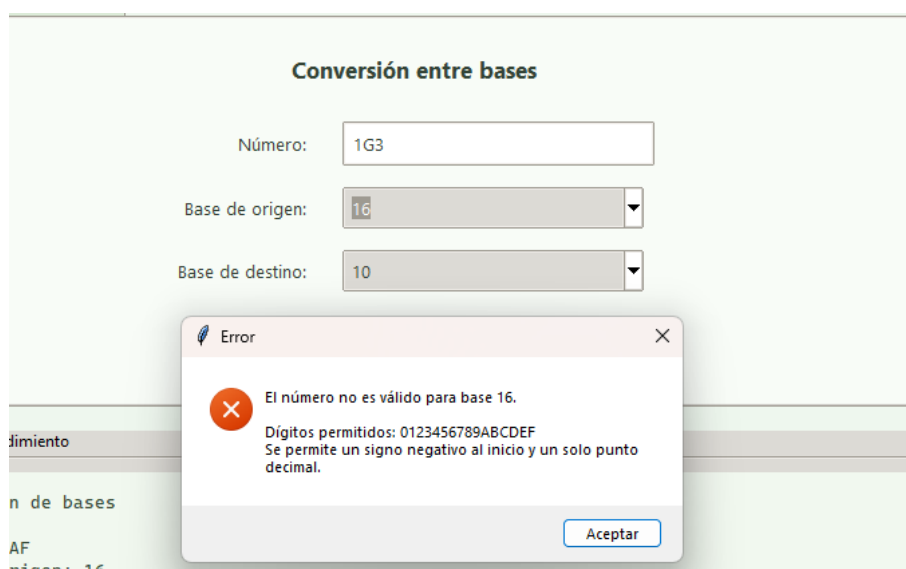


Figura 5: Validación de una letra inválida: G no es un dígito válido en base 16.

### 3.2 Casos del sumador

La Figura 6 muestra la suma de dos positivos sin overflow; la Figura 7 suma un positivo y un negativo; la Figura 8 suma dos negativos. Las Figuras 9 y 10 muestran el desbordamiento al sumar dos positivos y dos negativos, respectivamente; la Figura 11 el error al ingresar un valor fuera del intervalo y la Figura 12 el caso límite  $-32$ .

**Sumador binario con signo de 6 bits**

Primer decimal (-32 a 31):

Segundo decimal (-32 a 31):

**Calcular**

---

**Resultado y procedimiento**

```
Suma binaria:
  001010
+ 000111
-----
  010001

Resultado binario final: 010001
Interpretación decimal: 17

Acarreo que entra al signo: 0
Acarreo que sale del signo: 0
Overflow: NO
Los acarreos de entrada y salida del signo son iguales.
```

Figura 6: Suma de dos positivos sin overflow:  $10 + 7 = 17$ .

**Sumador binario con signo de 6 bits**

Primer decimal (-32 a 31):

Segundo decimal (-32 a 31):

**Calcular**

---

**Resultado y procedimiento**

```
Suma binaria:
  010011
+ 101010
-----
  111101

Resultado binario final: 111101
Interpretación decimal: -3

Acarreo que entra al signo: 0
Acarreo que sale del signo: 0
Overflow: NO
Los acarreos de entrada y salida del signo son iguales.
```

Figura 7: Suma de un positivo y un negativo:  $19 + (-22) = -3$ .



Conversión

Sumador

### Sumador binario con signo de 6 bits

Primer decimal (-32 a 31):

Segundo decimal (-32 a 31):

---

**Resultado y procedimiento**

```

Suma binaria:
  111000
+ 111011
-----
  110011

Resultado binario final: 110011
Interpretación decimal: -13

Acarreo que entra al signo: 1
Acarreo que sale del signo: 1
Overflow: NO
Los acarreos de entrada y salida del signo son iguales.
  
```

Figura 8: Suma de dos negativos:  $-8 + (-5) = -13$ .

Conversion

Sumador

### Sumador binario con signo de 6 bits

Primer decimal (-32 a 31):

Segundo decimal (-32 a 31):

---

**Resultado y procedimiento**

```

+ 010100
-----
 101000

Resultado binario final: 101000
Interpretación decimal: -24

Acarreo que entra al signo: 1
Acarreo que sale del signo: 0
Overflow: Sí
Los acarreos son diferentes, por lo tanto ocurrió desbordamiento.
El patrón de 6 bits se muestra, pero su interpretación decimal no representa la suma matemática.
  
```

Figura 9: Overflow al sumar dos positivos:  $20 + 20 = 40$  excede el rango de  $-32$  a  $31$ .

Conversión

Sumador

### Sumador binario con signo de 6 bits

Primer decimal (-32 a 31):

Segundo decimal (-32 a 31):

#### Resultado y procedimiento

```
  101100
+ 101100
-----
  011000

Resultado binario final: 011000
Interpretación decimal: 24

Acarreo que entra al signo: 0
Acarreo que sale del signo: 1
Overflow: SÍ
Los acarreos son diferentes, por lo tanto ocurrió desbordamiento.
El patrón de 6 bits se muestra, pero su interpretación decimal no representa la suma matemática
```

Figura 10: Overflow al sumar dos negativos:  $-20 + (-20)$  excede el rango de  $-32$  a  $31$ .

Conversión

Sumador

### Sumador binario con signo de 6 bits

Primer decimal (-32 a 31):

Segundo decimal (-32 a 31):

#### Resultado y procedimiento

```
  101100
+ 101100
-----
```

Error

 Cada número debe estar en el rango de -32 a 31.

Figura 11: Validación de rango: 32 está fuera del intervalo de  $-32$  a  $31$ .

Conversión

Sumador

### Sumador binario con signo de 6 bits

Primer decimal (-32 a 31):

Segundo decimal (-32 a 31):

#### Resultado y procedimiento

```
Suma binaria:
 100000
+ 000001
-----
 100001

Resultado binario final: 100001
Interpretación decimal: -31

Acarreo que entra al signo: 0
Acarreo que sale del signo: 0
Overflow: NO
Los acarreos de entrada y salida del signo son iguales.
```

Figura 12: Caso límite  $-32$ : se muestra su representación especial  $100000_2$ ;  $-32 + 1 = -31$  sin overflow.